



Netease Asset Management System - Complete Documentation

Project Information

- **Name:** Netease
 - **Version:** 1.0.0
 - **Type:** Asset Management System
 - **Backend:** FastAPI + PostgreSQL
 - **Frontend:** HTML/CSS/JavaScript (Demo)
-

Project Structure

```
netease/
├── app/                # Main application code
│   ├── __init__.py    # Package initialization
│   └── main.py         # Application entry point
│
│   ├── core/          # Core functionality (shared across modules)
│   │   ├── __init__.py
│   │   ├── config.py  # Configuration settings
│   │   ├── database.py # Database connection & session
│   │   └── security.py # Authentication & password hashing
│   │
│   ├── models/        # Database models (SQLAlchemy ORM)
│   │   ├── __init__.py
│   │   ├── user.py    # User model & UserRole enum
│   │   └── login_log.py # Login logs model
│   │
│   ├── schemas/       # Pydantic schemas (validation & serialization)
│   │   ├── __init__.py
│   │   ├── auth.py    # Login, Token schemas
│   │   ├── user.py    # User CRUD schemas
│   │   └── log.py     # Log response schemas
│   │
│   └── modules/       # Feature modules (independent features)
│       ├── __init__.py
```

```

| | | └─ auth/          # Authentication module
| | |   | └─ __init__.py
| | |   | └─ router.py   # Login API endpoints
| | |   └─ service.py    # Authentication business logic
| | |
| | | └─ users/         # User management module
| | |   | └─ __init__.py
| | |   | └─ router.py   # User CRUD API endpoints
| | |   └─ service.py    # User management business logic
| | |
| | | └─ logs/          # Login logs module
| | |   | └─ __init__.py
| | |   | └─ router.py   # Logs API endpoints
| | |   └─ service.py    # Logs business logic (future)
| | |
| | └─ utils/           # Utility functions
| |   └─ __init__.py
| |
| └─ scripts/           # Helper scripts
|   └─ create_users.py   # Script to create initial users
|
| └─ tests/             # Tests (future)
|   └─ __init__.py
|
| └─ alembic/           # Database migrations (future)
|   └─ versions/
|
| └─ .env               # Environment variables (NOT in Git)
| └─ .gitignore         # Git ignore rules
| └─ requirements.txt    # Python dependencies
| └─ README.md          # Project readme

```

File Descriptions

Root Files

.env (Environment Variables)

Purpose: Store sensitive configuration (NOT in Git)

env

```
DATABASE_URL=postgresql://user:pass@host/db
SECRET_KEY=your-secret-key
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

.gitignore

Purpose: Tell Git which files to ignore

- Ignores: venv/, **pycache/**, .env, *.db

requirements.txt

Purpose: List all Python dependencies

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
sqlalchemy==2.0.23
psycopg2-binary==2.9.9
...
```

app/main.py (Entry Point)

Purpose: Application initialization and startup

What it does:

1. Creates FastAPI application
2. Configures CORS middleware
3. Creates database tables
4. Registers all module routers
5. Defines root endpoints

Code structure:

```
python
```

```
from fastapi import FastAPI
from app.modules.auth import router as auth_router
from app.modules.users import router as users_router
from app.modules.logs import router as logs_router

app = FastAPI(title="Netease")

# Register routes
app.include_router(auth_router.router, prefix="/auth")
app.include_router(users_router.router, prefix="/api/users")
app.include_router(logs_router.router, prefix="/api/logs")
```

How to run:

```
bash

python -m app.main
```

app/core/ (Core Functionality)

core/config.py

Purpose: Centralized configuration management

What it does:

- Loads environment variables from .env
- Provides settings object for entire app
- Defines default values

Key settings:

- `DATABASE_URL`: PostgreSQL connection string
- `SECRET_KEY`: JWT signing key
- `ACCESS_TOKEN_EXPIRE_MINUTES`: Token expiration time

Usage example:

```
python
```

```
from app.core.config import settings
print(settings.DATABASE_URL)
```

core/database.py

Purpose: Database connection management

What it does:

- Creates SQLAlchemy engine (connection to PostgreSQL)
- Creates SessionLocal (database session factory)
- Defines Base class for all models
- Provides `get_db()` dependency for routes

Key components:

```
python

engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Usage in routes:

```
python

@router.get("/users")
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()
```

core/security.py

Purpose: Security functions (passwords & JWT)

What it does:

- Password hashing with bcrypt
- Password verification
- JWT token creation and validation

Key functions:

```
python

get_password_hash(password)  # Hash password
verify_password(plain, hashed) # Verify password
create_access_token(data)    # Create JWT token
```

Usage example:

```
python

hashed = get_password_hash("password123")
is_valid = verify_password("password123", hashed)
token = create_access_token({"sub": "username"})
```

app/models/ (Database Models)

models/user.py

Purpose: User database model

What it defines:

- `UserRole` enum (admin, manager, user, guest)
- `User` table structure

Table structure:

```
sql
```

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  hashed_password VARCHAR(255) NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  role VARCHAR(20) NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Relationships:

- One user can have many login logs
- One user can own many assets (future)

models/login_log.py

Purpose: Login attempt logging

What it tracks:

- Every login attempt (success or failure)
- Username, IP address, user agent
- Timestamp and custom message

Table structure:

```
sql  
  
CREATE TABLE login_logs (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  success BOOLEAN NOT NULL,  
  ip_address VARCHAR(50),  
  user_agent TEXT,  
  message TEXT,  
  timestamp TIMESTAMP DEFAULT NOW()  
);
```

Purpose: Define data structures for API input/output

Why we need schemas:

1. **Validation:** Ensure data is correct before saving
2. **Serialization:** Convert database objects to JSON
3. **Documentation:** Auto-generate API docs
4. **Type safety:** Catch errors early

schemas/auth.py

```
python
```

```
class UserLogin(BaseModel):  
    username: str  
    password: str  
  
class Token(BaseModel):  
    access_token: str  
    token_type: str  
    role: str  
    username: str
```

schemas/user.py

```
python
```

```
class UserCreate(BaseModel):
    username: str
    email: EmailStr
    password: str
    role: str = "user"

class UserUpdate(BaseModel):
    username: Optional[str] = None
    email: Optional[EmailStr] = None
    password: Optional[str] = None
    role: Optional[str] = None

class UserResponse(BaseModel):
    id: int
    username: str
    email: str
    role: str
    is_active: bool
    created_at: datetime
```

app/modules/ (Feature Modules)

Architecture Pattern: Each module is independent

Structure of each module:

```
module_name/
├── __init__.py    # Export router
├── router.py      # API endpoints (HTTP layer)
└── service.py     # Business logic (Application layer)
```

Why this separation?

- **router.py:** Handles HTTP requests/responses only
- **service.py:** Contains business logic, database operations
- **Benefit:** Easy to test, maintain, and reuse

modules/auth/

Purpose: User authentication

router.py:

```
python

@router.post("/login")
def login(credentials: UserLogin, request: Request, db: Session):
    # 1. Get IP and user agent from request
    # 2. Call service to authenticate
    # 3. Log the attempt
    # 4. Return JWT token or error
```

service.py:

```
python

class AuthService:
    def authenticate_user(username, password):
        # 1. Find user in database
        # 2. Verify password
        # 3. Return user or None
```

API Endpoints:

- `POST /auth/login` - User login
-

modules/users/

Purpose: User management (CRUD)

router.py:

```
python

@router.get("/")      # List all users
@router.get("/{id}")   # Get one user
@router.post("/")      # Create user
@router.put("/{id}")   # Update user
@router.delete("/{id}") # Delete user
@router.get("/search/") # Search users
```

service.py:

```
python
```

```
class UserService:
    def get_all_users()    # List users
    def get_user_by_id(id) # Get one
    def create_user(data)  # Create
    def update_user(id, data) # Update
    def delete_user(id)    # Delete
    def search_users(query) # Search
```

Business logic includes:

- Check for duplicate usernames/emails
- Validate role values
- Hash passwords before saving
- Prevent deleting your own account

modules/logs/

Purpose: View login logs and statistics

router.py:

```
python

@router.get("/")    # All logs
@router.get("/user/{username}") # User-specific logs
@router.get("/stats") # Statistics
```

What statistics provides:

- Total login attempts
- Successful vs failed count
- Success rate percentage
- Recent successful logins

scripts/create_users.py

Purpose: Create initial admin users for testing

What it does:

1. Connects to database
2. Creates default users (admin, manager, user1)
3. Hashes passwords
4. Checks for duplicates

How to use:

```
bash
```

```
python scripts/create_users.py
```

Request Flow (How Everything Works Together)

Example: User Login Request

1. Client (Browser/Frontend)
 - ↳ POST /auth/login
 - Body: {"username": "sina", "password": "1234"}
2. main.py
 - ↳ Routes request to auth module
3. modules/auth/router.py
 - ↳ Receives request
 - ↳ Extracts username, password, IP, user agent
 - ↳ Calls AuthService.authenticate_user()
4. modules/auth/service.py
 - ↳ Queries database for user
 - ↳ Uses core/security.py to verify password
 - ↳ Returns User object or None
5. Back to router.py
 - ↳ If success:
 - Log success to login_logs table
 - Create JWT token using core/security.py
 - Return token to client
 - ↳ If failure:
 - Log failure to login_logs table
 - Return 401 error

6. Database Operations

- ↳ Uses core/database.py session
- ↳ Queries models/user.py
- ↳ Inserts to models/login_log.py

7. Response to Client

- ↳ schemas/auth.py Token format
- ↳ JSON: {"access_token": "...", "role": "admin", ...}

Database Schema

Current Tables:

users

```
sql
id          SERIAL PRIMARY KEY
username    VARCHAR(50) UNIQUE NOT NULL
email       VARCHAR(255) UNIQUE NOT NULL
hashed_password VARCHAR(255) NOT NULL
is_active   BOOLEAN DEFAULT TRUE
role        ENUM('admin','manager','user','guest')
created_at  TIMESTAMP DEFAULT NOW()
```

login_logs

```
sql
id          SERIAL PRIMARY KEY
username    VARCHAR(50) NOT NULL
success     BOOLEAN NOT NULL
ip_address  VARCHAR(50)
user_agent  TEXT
message     TEXT
timestamp   TIMESTAMP DEFAULT NOW()
```

Future Tables (Asset Management):

- asset_inventory (23 columns)
- asset_types
- asset_owners

- asset_locations
 - asset_dependencies
 - asset_security_status
 - • 7 enum tables
-

Security Features

Password Security:

- ☒ Bcrypt hashing (one-way encryption)
- ☒ Automatic salt generation
- ☒ 72-byte limit (bcrypt requirement)

JWT Tokens:

- ☒ HS256 algorithm
- ☒ 30-minute expiration
- ☒ Contains username and role
- ☒ Signed with SECRET_KEY

Input Validation:

- ☒ Pydantic schemas validate all inputs
- ☒ Email format validation
- ☒ Username length constraints (3-50 chars)
- ☒ Password minimum length (4 chars)

Logging:

- ☒ All login attempts logged
 - ☒ IP address tracking
 - ☒ User agent tracking
 - ☒ Success/failure status
-

API Endpoints (Current)

Authentication

POST /auth/login

Body: {username, password}

Response: {access_token, token_type, role, username}

User Management

GET /api/users/ List all users

GET /api/users/{id} Get user details

POST /api/users/ Create user

PUT /api/users/{id} Update user

DELETE /api/users/{id} Delete user

GET /api/users/search?q= Search users

Logs

GET /api/logs/ All logs

GET /api/logs/?success_only=true Successful only

GET /api/logs/?success_only=false Failed only

GET /api/logs/user/{username} User-specific logs

GET /api/logs/stats Statistics

System

GET / Root endpoint

GET /health Health check

GET /docs Auto-generated API docs



Development Workflow

1. Starting Development Server

```
bash
```

```
cd ~/main_app/netease
```

```
source venv/bin/activate
```

```
python -m app.main
```

2. Making Changes

```
bash

# Edit files
nano app/models/user.py

# Git workflow
git status
git add .
git commit -m "Description of changes"
git push
```

3. Database Changes

```
bash

# For now: drop and recreate
sudo -u postgres psql
DROP DATABASE netease_db;
CREATE DATABASE netease_db;
\q

# Recreate tables
python scripts/create_users.py
```

4. Testing

```
bash

# Manual testing
curl http://localhost:8000/health

# API docs
http://localhost:8000/docs
```

Key Concepts & Patterns

1. Layered Architecture

```
Presentation Layer (router.py)
↓
Business Logic Layer (service.py)
```

↓
Data Access Layer (models.py)
↓
Database (PostgreSQL)

2. Dependency Injection

```
python  
  
def endpoint(db: Session = Depends(get_db)):  
    # db is automatically provided
```

3. Separation of Concerns

- **Models:** Database structure
- **Schemas:** API data structure
- **Services:** Business logic
- **Routers:** HTTP handling

4. DRY (Don't Repeat Yourself)

- Common code in `core/`
- Reusable functions in `utils/`
- Shared schemas in `schemas/`

Next Steps (Asset Management)

Phase 1: Database Models

- Create 13 new models for assets
- Define relationships
- Create enums

Phase 2: Schemas

- Input/output schemas for all models
- Validation rules

Phase 3: Services

- Asset CRUD operations

- Dependency checking
- CSV import/export

Phase 4: APIs

- RESTful endpoints
- Filtering & pagination
- Search functionality

Phase 5: Auto Discovery

- nmap integration
 - Network scanning
 - Automatic asset detection
-

Learning Resources

SQLAlchemy (ORM)

- Official docs: <https://docs.sqlalchemy.org/>
- Relationships: One-to-Many, Many-to-Many
- Querying: Filter, Join, Eager Loading

FastAPI

- Official docs: <https://fastapi.tiangolo.com/>
- Dependency Injection
- Background Tasks
- WebSockets (future)

Pydantic

- Data validation
- Settings management
- Serialization

PostgreSQL

- Foreign Keys

- Indexes
 - Constraints
 - Transactions
-

Contributing Guidelines (Future)

Code Style

- Use English for code and comments
- Follow PEP 8 (Python style guide)
- Descriptive variable names
- Add docstrings to functions

Git Workflow

```
bash

# Create feature branch
git checkout -b feature/asset-management

# Make changes and commit
git add .
git commit -m "Add asset inventory model"

# Push to GitHub
git push origin feature/asset-management
```

Testing (Future)

- Unit tests for services
 - Integration tests for APIs
 - Test coverage > 80%
-

Troubleshooting

Common Issues:

"ModuleNotFoundError: No module named 'app'"

Solution: Run from project root: `python -m app.main`

"permission denied for schema public"

Solution: Grant PostgreSQL permissions:

```
sql

GRANT ALL ON SCHEMA public TO netease;
GRANT CREATE ON SCHEMA public TO netease;
```

"Port 8000 already in use"

Solution: Kill existing process:

```
bash

sudo lsof -i :8000
sudo kill -9 <PID>
```

Git push rejected

Solution: Pull first or force push:

```
bash

git pull origin main --allow-unrelated-histories
# or
git push --force
```

Contact & Support

Developer: [Your Name] **Project Repository:** <https://github.com/SinaBimesl/netease> **Documentation**

Date: November 2024

This documentation is a living document and will be updated as the project evolves.